HTTPS & TLS 1.3 Handshake

ECDHE, forward secrecy and 0-RTT

Watch the 5-min video

In one line: HTTPS is HTTP inside TLS; the TLS 1.3 handshake agrees on fresh secret keys with ECDHE and proves the server's identity with a certificate, all in one round trip, then switches to fast symmetric encryption.

1 What TLS guarantees

Every padlock in the browser means a TLS handshake just happened, usually in tens of milliseconds. **HTTPS** is plain HTTP carried inside a **TLS** connection (port 443). TLS adds three guarantees:

- **Confidentiality:** nobody on the network can read the traffic.
- **Integrity:** nobody can change a byte without the receiver noticing.
- **Authentication:** the server proves who it is with a certificate (the next lesson).

2 The TLS 1.3 handshake, step by step

One round trip both agrees on keys and proves the server's identity:

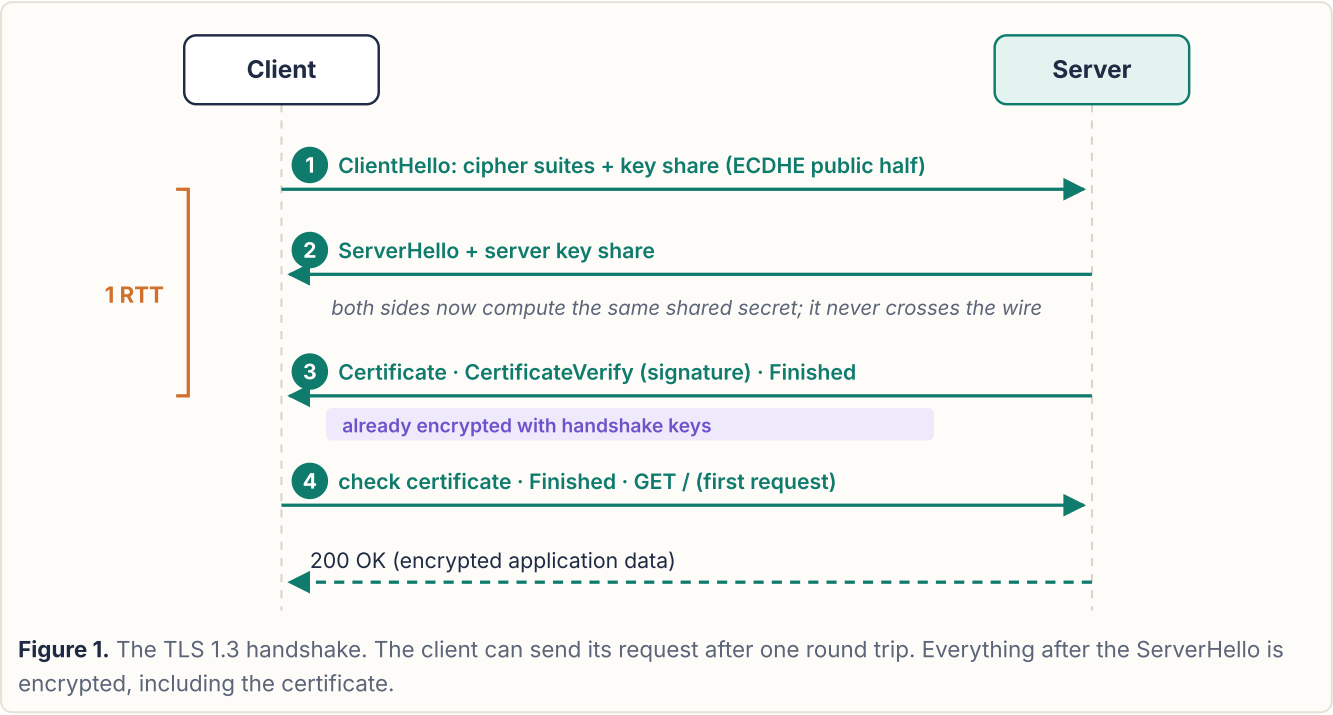


Figure 1. The TLS 1.3 handshake. The client can send its request after one round trip. Everything after the ServerHello is encrypted, including the certificate.

1. **ClientHello.** The client lists the cipher suites it supports and sends a **key share**: its public half of an **ECDHE** (elliptic-curve Diffie-Hellman, ephemeral) exchange, usually on curve X25519.
2. **ServerHello.** The server picks a suite and sends its own key share. From the two shares, each side computes the **same shared secret**. The secret itself is never sent.
3. **Server's encrypted flight.** Already encrypted, the server sends its **Certificate**, a **CertificateVerify** (a signature over the handshake made with its private key, proving it owns the certificate) and **Finished** (a MAC over the whole handshake, so nothing was tampered with).

4. **Client finishes.** The client checks the certificate and signature, sends its own Finished and, in the same flight, its first HTTP request.

3 Why TLS 1.3 is faster and safer than 1.2

TLS 1.2 needed **two round trips** before the first request, and still allowed old, weak options (RSA key exchange, CBC modes, RC4, SHA-1). TLS 1.3 needs **one**, removed the legacy algorithms (only five cipher suites remain, all AEAD, such as `TLS_AES_128_GCM_SHA256`), and made forward secrecy mandatory.



Figure 2. Time to the first response on a 100 ms round-trip link, counting the TCP handshake. TLS 1.3 saves 100 ms on every new connection; 0-RTT resumption saves another 100 ms.

Expensive once, cheap after. Public-key crypto (key exchange and signature) runs only during the handshake. Then both sides derive **session keys** from the shared secret and switch to fast **symmetric** encryption such as AES-GCM or ChaCha20-Poly1305, which modern CPUs do at gigabytes per second.

Resumption. After a handshake the server can hand the client a session ticket. Next time, the client offers it as a **pre-shared key (PSK)** and the server skips sending and checking the certificate. TLS 1.3 also allows **0-RTT**: the client sends *early data* (say, a GET) in its very first message. The catch: an attacker who copies that first message can **replay** it, and the server cannot tell. So accept 0-RTT only for **idempotent** requests such as GETs, never for "pay €50".

4 Forward secrecy

Suppose an attacker records all your encrypted traffic today and steals the server's private key next year. With the old **RSA key exchange**, the client encrypted the session secret with the server's long-term public key, so the stolen key unlocks every recording. With **ECDHE**, each session uses fresh **ephemeral** keys that are thrown away afterwards; the long-term private key only *signs* the handshake. Stealing it later decrypts nothing from the past. That property is **forward secrecy**.

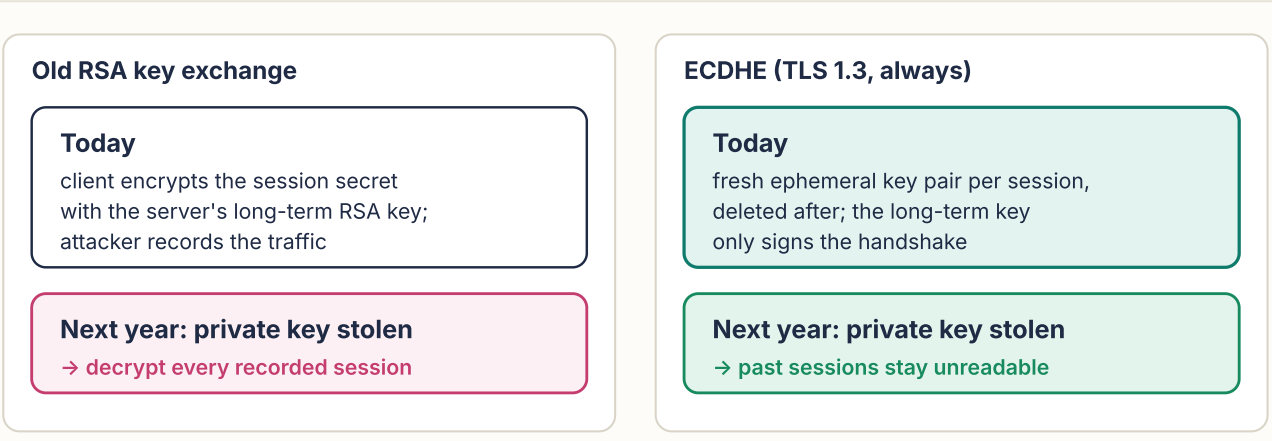


Figure 3. With RSA key exchange one stolen key exposes years of recorded traffic. With ECDHE, a stolen key lets an attacker impersonate the server from then on, but not read the past.

5 Where TLS ends in your architecture

Most often TLS **terminates at the load balancer**. It holds the public certificate, decrypts, and can route by path, header or cookie, add headers like `X-Forwarded-Proto`, and run a WAF. It then **re-encrypts** to the backends over a second TLS connection, so traffic stays encrypted inside your network too, as zero-trust and compliance rules (PCI DSS, HIPAA) often require. In **passthrough** mode the load balancer forwards raw encrypted bytes (layer 4) and the backends terminate TLS themselves: true end-to-end encryption, but the load balancer cannot see HTTP, so no path routing.

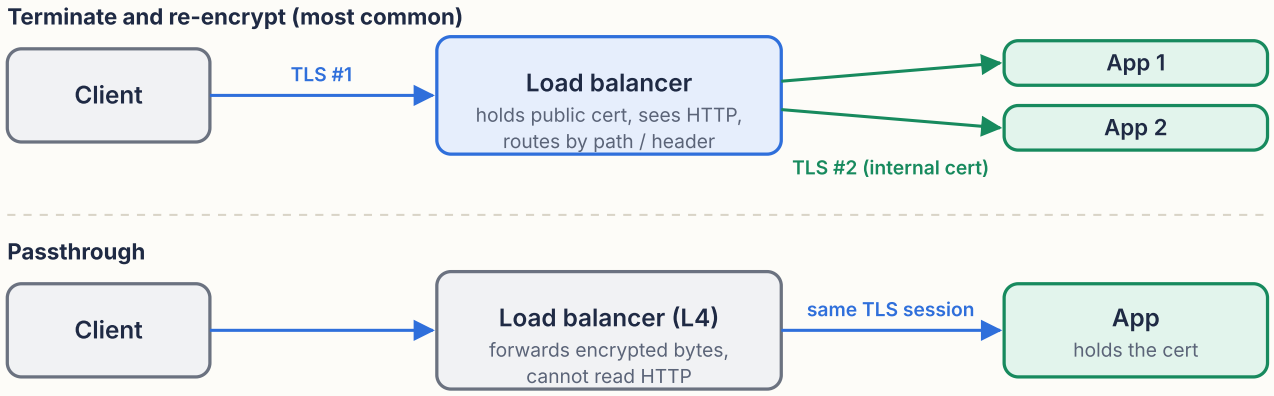


Figure 4. Top: two separate TLS connections, with the load balancer able to read and route HTTP. Bottom: one TLS session from client to app; the load balancer only sees bytes.

WORKED EXAMPLE

A mobile API: 100 ms round trip, users open a new connection on every app launch. TCP + TLS 1.2 + request = 4 round trips ≈ **400 ms** to the first byte. Moving to TLS 1.3: 3 round trips ≈ **300 ms**. Returning users with resumption skip certificate checks; enabling 0-RTT for the idempotent `GET /feed` brings it to ≈ **200 ms**. `POST /orders` still waits for the full handshake. (HTTP/3 folds the TCP step in too: lesson SD-N-023.)

COMMON MISTAKES

- Accepting 0-RTT early data for POSTs or payments: it can be replayed.
- Leaving TLS 1.0 / 1.1 or RSA key exchange enabled "for old clients": it removes forward secrecy for everyone who negotiates down.
- Terminating at the load balancer and sending plain HTTP inside, then assuming the internal network is safe.

6 Key terms

TLS: encrypts and authenticates a connection

CertificateVerify: signature proving the server holds the private key

Forward secrecy: a stolen long-term key can't decrypt past traffic

ECDHE: ephemeral elliptic-curve Diffie-Hellman key exchange

Finished: MAC over the handshake transcript

PSK / resumption: reuse a previous session, skip cert work

Key share: one side's public half of ECDHE

AES-GCM: fast symmetric AEAD cipher for bulk data

0-RTT: data in the first message; replayable

7 Interview questions

Q1. Walk through a TLS 1.3 handshake. BASIC

Client sends ClientHello with cipher suites and an ECDHE key share. Server replies with ServerHello and its key share; both derive the same secret. The server sends, encrypted, its certificate, a CertificateVerify signature and Finished. The client verifies, sends Finished and its first request: one round trip.

Q2. What is forward secrecy? BASIC

Each session uses ephemeral ECDHE keys that are deleted afterwards, and the long-term key only signs. So stealing the server's private key later cannot decrypt traffic recorded in the past. TLS 1.3 makes it mandatory.

Q3. Why use symmetric encryption after the handshake? BASIC

Public-key operations cost around a millisecond of CPU each; AES-GCM with hardware support encrypts several GB/s per core. So the handshake pays for asymmetric crypto once, then bulk data uses symmetric session keys.

Q4. Your app has 100 ms RTT and users complain about slow first loads. What does TLS 1.3 buy you? SCENARIO

One round trip less per new connection: about 400 ms to 300 ms counting TCP. Add session resumption, and 0-RTT for idempotent GETs, to reach about 200 ms. HTTP/3 would also merge the TCP handshake.

Q5. Security asks for encryption inside the data centre, but you route by URL path at the load balancer. What do you do? SCENARIO

Terminate TLS at the load balancer (it needs HTTP to route), then re-encrypt to the backends with internal certificates, or mTLS through a service mesh. Passthrough would block path routing.

Q6. Should you enable 0-RTT for your whole API? SCENARIO

No. Early data can be replayed by anyone who captured it, and the server can't detect it on its own. Allow it only for idempotent, safe requests (GETs) and reject or delay early data for POST, PUT and payments.

Q7. If TLS 1.3 encrypts the certificate, can a network observer still see which site you visit? TRICKY

Usually yes. The destination IP is visible, and the SNI hostname in the ClientHello is plaintext unless Encrypted Client Hello (ECH) is used. TLS 1.3 hides the certificate, not the destination.

Q8. An attacker steals your TLS private key today. What can they do with TLS 1.3? TRICKY

Impersonate your server to clients going forward (until the certificate is revoked and replaced), but not decrypt previously recorded sessions, thanks to ephemeral ECDHE. Rotate the key, revoke the certificate, and check for misuse.

Next: SD-N-022 · Certificates and PKI: why your browser trusts a site it has never met backendroadmap.com